Методы строк в Python



Что такое метод строки?

Метод строки — это встроенная операция Python, которая позволяет выполнять определённые действия с текстом.

Методы вызываются через точку . после строки или переменной.

string_methods.py

```
text = "hello python"
print(text.upper())
# HELLO PYTHON
```

Важно: строки в Python неизменяемы. Методы не изменяют исходную строку, а возвращают новый результат.

```
text = "hello"
text.upper()
print(text)  # hello
```

Чтобы сохранить результат, нужно присвоить его переменной:

```
text = text.upper()
```

Метод capitalize()

Назначение: преобразует первый символ строки в верхний регистр, а остальные символы — в нижний.

Синтаксис:

```
string.capitalize()
```

Пример:

```
text = "hELLO PYTHON"
print(text.capitalize())
# Hello python
```

Задание №1

Пользователь вводит название своего любимого города в произвольном регистре. Выведите название так, чтобы первая буква была заглавной, а остальные — строчными.

Метод swapcase()

Назначение: меняет регистр букв на противоположный: заглавные становятся строчными, а строчные — заглавными.

Синтаксис:

```
string.swapcase()
```

Пример:

```
text = "Hello PYTHON"
print(text.swapcase())
# hELLO python
```

Задание №2

Пользователь вводит текст, например "PyThOn Is FuN". Измените регистр всех букв на противоположный.



Метод title()

Назначение: преобразует первую букву каждого слова в верхний регистр, остальные — в нижний.

Синтаксис метода

Синтаксис:

```
string.title()
```

Демонстрация вызова

Пример:

```
text = "hello python world"
print(text.title())
# Hello Python World
```

Задание №3

Пользователь вводит своё имя, фамилию и отчество маленькими буквами. Выведите ФИО так, чтобы каждое слово начиналось с заглавной буквы.

Метод lower()

Назначение: преобразует все буквы строки в нижний регистр.

Синтаксис

Синтаксис:

```
string.lower()
```

Пример

Пример:

```
text = "HELLO Python"
print(text.lower())
# hello python
```

Задание №4

Напишите программу, которая запрашивает у пользователя ответ на вопрос: «Вы любите Python?».

Пользователь может написать "ДА", "Да" или "да". Программа должна преобразовать ответ в нижний регистр и проверить, равен ли он строке "да".

Если равен, вывести "Отлично!", иначе — "Понятно!".

Метод upper()

Назначение: преобразует все буквы строки в верхний регистр.

Синтаксис:

```
string.upper()
```

Пример:

```
text = "Hello Python"
print(text.upper())
# HELLO PYTHON
```

Задание №5

Пользователь вводит название страны. Программа должна вывести его заглавными буквами.

Дополнительно: выведите сообщение "Добро пожаловать в РОССИЮ!", используя введённое название страны.



Метод count()

Назначение: подсчитывает количество неперекрывающихся вхождений указанного символа или подстроки.

Синтаксис:

```
string.count(substring)
```

Пример:

```
text = "banana"
print(text.count("a"))   # 3
print(text.count("an"))  # 2
print(text.count("z"))   # 0
```

Метод учитывает регистр символов.

Задание №6

Пользователь вводит предложение. Программа должна подсчитать, сколько раз в нём встречается буква "a", независимо от регистра.

Подсказка: используйте lower() и count().

Метод startswith()

Назначение: проверяет, начинается ли строка с указанной подстроки.

Возвращает логическое значение True или False.

Синтаксис:

```
string.startswith(prefix)
```

Пример:

```
text = "Python programming"
print(text.startswith("Python"))  # True
print(text.startswith("Java"))    #
False
```

Задание №7

Пользователь вводит адрес сайта. Проверьте, начинается ли он с "https://".

Если начинается, выведите "HTTPS используется", иначе — "HTTPS не обнаружен".

Метод endswith()

Назначение: проверяет, заканчивается ли строка указанной подстрокой.

Синтаксис:

```
string.endswith(suffix)
```

Пример:

```
filename = "program.py"
print(filename.endswith(".py"))   # True
print(filename.endswith(".txt"))  #
False
```

Задание №8

Пользователь вводит имя файла. Определите, является ли файл Python-скриптом по расширению .py.

Дополнительно: программа должна правильно обрабатывать имена файлов "MAIN.PY" и "main.py".

Метод find()

Назначение: ищет первое вхождение подстроки и возвращает индекс её начала.

Если подстрока не найдена, возвращает -1.

Синтаксис:

```
string.find(substring)
```

Пример:

```
text = "Hello Python"
print(text.find("Python"))  # 6
print(text.find("o"))       # 4
print(text.find("Java"))    # -1
```

Задание №9

Пользователь вводит адрес электронной почты. Найдите позицию символа @. Если символ отсутствует, выведите сообщение "Символ @ не найден".

Метод rfind()

Назначение: ищет последнее вхождение подстроки, начиная поиск с правой стороны строки.

Возвращает индекс начала найденной подстроки или -1, если она отсутствует.

Синтаксис:

```
string.rfind(substring)
```

Пример:

```
text = "Python is great. I love Python"
print(text.find("Python"))   # 0
print(text.rfind("Python"))  # 24
```

Задание №10

Пользователь вводит имя файла, например:

```
archive.backup.zip
```

Найдите индекс последней точки в имени файла.

Дополнительно: используя срез строки, выведите расширение файла.

Метод index()

Назначение: ищет первое вхождение подстроки и возвращает её индекс.

В отличие от find(), если подстрока не найдена, метод index() вызывает ошибку ValueError.

Синтаксис:

```
string.index(substring)
```

Пример:

```
text = "Hello Python"
print(text.index("Python"))  # 6
print(text.index("Java"))    #
ValueError
```

Задание №11

Дана строка:

```
text = "Student: Alex, Age: 18"
```

Найдите индекс первого символа : с помощью index(). Затем выведите часть строки, которая расположена до двоеточия.

Метод rindex()

Назначение: находит последнее вхождение подстроки и возвращает его индекс.

Если подстрока отсутствует, вызывает ValueError.

Синтаксис:

```
string.rindex(substring)
```

Пример:

```
text = "one two one"
print(text.index("one"))   # 0
print(text.rindex("one"))  # 8
```

Задание №12

Дана строка:

```
path = "documents/python/lesson.py"
```

Найдите индекс последнего символа / с помощью rindex(). Используя срез, выведите только имя файла: "lesson.py".

Метод strip()

Назначение: удаляет пробельные символы в начале и конце строки.

Синтаксис:

```
string.strip()
```

Пример:

```
text = "   Hello Python   "
print(text.strip())
# Hello Python
```

Метод также позволяет указывать символы, которые нужно удалить по краям:

```
text = "***Python***"
print(text.strip("*"))
# Python
```

Задание №13

Пользователь вводит логин, который может содержать лишние пробелы в начале и конце. Удалите пробелы и проверьте, равен ли результат строке "admin".

Метод lstrip()

Назначение: удаляет пробельные или указанные символы только с левой стороны строки.

Синтаксис:

```
string.lstrip()
```

Пример:

```
text = "   Python   "
print(text.lstrip())
# "Python   "
```

Задание №14

Дана строка:

```
text = "###Python"
```

Удалите все символы # в начале строки.

Ожидаемый результат:

Python

Метод rstrip()

Назначение: удаляет пробельные или указанные символы только с правой стороны строки.

Синтаксис:

```
string.rstrip()
```

Пример:

```
text = "Python!!!"
print(text.rstrip("!"))
# Python
```

Задание №15

Пользователь вводит сообщение, которое может заканчиваться несколькими восклицательными знаками. Например:

```
Привет!!!!!
```

Удалите все восклицательные знаки в конце строки.

Метод replace()

Назначение: заменяет одну подстроку другой.

Синтаксис:

```
string.replace(old, new)
```

Пример:

```
text = "I love Java"
print(text.replace("Java", "Python"))
# I love Python
```

Можно ограничить количество замен:

```
text = "cat cat cat"
print(text.replace("cat", "dog", 2))
# dog dog cat
```

Задание №16

Пользователь вводит предложение.

Замените все пробелы на символ _.

Дополнительно: замените все точки . на восклицательные знаки !.

Метод isalnum()

Назначение: проверяет, состоит ли строка только из букв и цифр и содержит ли хотя бы один символ.

Возвращает True или False.

Синтаксис:

```
string.isalnum()
```

Пример:

```
print("Python123".isalnum())  # True
print("Python 123".isalnum()) # False
print("Python!".isalnum())    # False
```

Задание №17

Пользователь вводит логин.

Проверьте, состоит ли он только из букв и цифр.

Если да, выведите "Логин допустим", иначе — "Недопустимые символы".

Метод isalpha()

Назначение: проверяет, состоит ли строка только из букв и содержит ли хотя бы один символ.

Синтаксис:

```
string.isalpha()
```

Пример:

```
print("Python".isalpha())   # True
print("Python3".isalpha())  # False
print("Hello World".isalpha()) # False
```

Задание №18

Пользователь вводит своё имя.

Проверьте, содержит ли оно только буквы.

Если присутствуют цифры, пробелы или специальные символы, выведите сообщение об ошибке.

Метод isdigit()

Назначение: проверяет, состоит ли строка только из цифровых символов и содержит ли хотя бы один символ.

Синтаксис:

```
string.isdigit()
```

Пример:

```
print("12345".isdigit())  # True
print("123a".isdigit())   # False
print("12.5".isdigit())   # False
```

Задание №19

Пользователь вводит возраст.

Проверьте, состоит ли введённое значение только из цифр.

Если да, преобразуйте строку в число с помощью int() и проверьте, исполнилось ли пользователю 18 лет.

Метод islower()

Назначение: проверяет, находятся ли все буквы строки в нижнем регистре.

Возвращает True, если в строке есть хотя бы одна буква и нет букв верхнего регистра.

Синтаксис:

```
string.islower()
```

Пример:

```
print("hello".islower())     # True
print("Hello".islower())     # False
print("hello123".islower())  # True
```

Задание №20

Пользователь вводит слово.

Если все буквы строчные, выведите "Нижний регистр".

В противном случае преобразуйте слово в нижний регистр и выведите результат.

Метод isupper()

Назначение: проверяет, находятся ли все буквы строки в верхнем регистре.

Возвращает True, если есть хотя бы одна буква и нет строчных букв.

Синтаксис:

```
string.isupper()
```

Пример:

```
print("HELLO".isupper())     # True
print("Hello".isupper())     # False
print("HELLO123".isupper())  # True
```

Задание №21

Пользователь вводит сообщение.

Проверьте, написано ли оно полностью заглавными буквами.

Если да, выведите "Не кричите!".

Иначе выведите "Обычное сообщение".

Метод isspace()

Назначение: проверяет, состоит ли строка только из пробельных символов.

К пробельным символам относятся обычный пробел, табуляция \t, перевод строки \n и другие.

Синтаксис:

```
string.isspace()
```

Пример:

```
print("   ".isspace())    # True
print(" \t\n".isspace())  # True
print("Hello".isspace())  # False
print("".isspace())       # False
```

Задание №22

Пользователь вводит сообщение.

Если сообщение состоит только из пробелов, выведите "Сообщение не должно состоять из пробелов".

Если строка пустая, выведите "Вы ничего не ввели".

В остальных случаях выведите сообщение пользователя.

Итоговая практическая работа

Мини-проект: «Обработка данных пользователя»

Напишите программу, которая последовательно запрашивает у пользователя имя, логин, возраст, адрес электронной почты и сообщение.

Программа должна:

1. Имя: удалить лишние пробелы, проверить, что имя состоит только из букв, и преобразовать первую букву в заглавную.
2. Логин: проверить, что логин состоит только из букв и цифр.
3. Возраст: проверить, что введены только цифры, затем определить, является ли пользователь совершеннолетним.
4. Email: проверить наличие символа @, найти его индекс и вывести часть адреса до @.
5. Сообщение: удалить пробелы по краям, проверить, что оно не пустое, и подсчитать количество букв "а" без учёта регистра.